

Algorithms for the Longest Increasing Back-and-Forth Subsequence Problem

Analysis by Dark and Mysterious TA

October 9, 2025

Abstract

This document presents four distinct algorithms to solve the Longest Increasing Back-and-Forth Subsequence (LIBFS) problem. For each algorithm, we provide a detailed description, pseudocode, a mathematically rigorous proof of correctness, and a formal analysis of its time and space complexity.

1 Problem Definition

Let $A[1..n]$ be an array of positive integers. An **increasing back-and-forth subsequence** is a sequence of indices $I[1..\ell]$ satisfying the following properties:

1. $1 \leq I[j] \leq n$ for all j .
2. $A[I[j]] < A[I[j+1]]$ for all $1 \leq j < \ell$ (Strictly Increasing Values).
3. If j is even, then $I[j+1] > I[j]$ (Move Right).
4. If j is odd, then $I[j+1] < I[j]$ (Move Left).

The task is to design an algorithm to compute the maximum possible length ℓ for a given array A .

2 Dynamic Programming with Sorted Input

2.1 Algorithm Description

This approach resolves the problem's complex dependencies by processing elements in increasing order of their **values**, not their indices. We create pairs of $(value, index)$ and sort them. This guarantees that when calculating the LIBFS for an element $P[k]$, the solutions for all valid predecessors (which must have a smaller value) have already been computed. We use two DP arrays, $OddLen[k]$ and $EvenLen[k]$, corresponding to the k -th element in the sorted list.

2.2 Pseudocode

2.3 Proof of Correctness

Proof. The proof is by induction on k , the position in the sorted list of pairs P .

Claim: For any $k \in [1, n]$, after the k -th iteration of the outer loop, $OddLen[k]$ and $EvenLen[k]$ store the correct maximum lengths of all odd and even LIBFS ending at element $P[k]$, respectively.

Base Case ($k = 1$): The algorithm sets $OddLen[1] = 1$ and $EvenLen[1] = 0$. This is correct, as the first element in the sorted value list can only form a subsequence of length 1 (odd).

Algorithm 1 DP with Sorted Input ($O(n^2)$)

```
1: procedure SOLVESORTEDDP( $A, n$ )
2:   if  $n = 0$  then return 0
3:   end if
4:   Let  $P$  be a list of pairs  $(A[i], i)$  for  $i = 1 \dots n$ .
5:   Sort  $P$  primarily by value, secondarily by index, in ascending order.
6:   Initialize arrays OddLen $[1 \dots n]$  and EvenLen $[1 \dots n]$ .
7:   for  $k \leftarrow 1$  to  $n$  do
8:     OddLen $[k] \leftarrow 1$ 
9:     EvenLen $[k] \leftarrow 0$ 
10:  end for
11:  for  $k \leftarrow 1$  to  $n$  do
12:    Let  $(v_k, p_k) \leftarrow P[k]$ 
13:    for  $j \leftarrow 1$  to  $k - 1$  do
14:      Let  $(v_j, p_j) \leftarrow P[j]$ 
15:      if  $v_j < v_k$  then
16:        if  $p_j < p_k$  then ▷ Right move creates an odd-length sequence
17:          if EvenLen $[j] > 0$  then
18:            OddLen $[k] \leftarrow \max(\text{OddLen}[k], \text{EvenLen}[j] + 1)$ 
19:          end if
20:        else if  $p_j > p_k$  then ▷ Left move creates an even-length sequence
21:          EvenLen $[k] \leftarrow \max(\text{EvenLen}[k], \text{OddLen}[j] + 1)$ 
22:        end if
23:      end if
24:    end for
25:  end for
26:  return  $\max(\text{OddLen} \cup \text{EvenLen})$ 
27: end procedure
```

Inductive Hypothesis (IH): Assume the claim is true for all integers $j < k$.

Inductive Step (for k): We must show the claim holds for k . To compute $\text{OddLen}[k]$, we seek to extend a previously computed even-length subsequence with a rightward move. A valid predecessor $P[j]$ must satisfy:

1. $j < k$ (it has already been processed). By sorting, this implies $v_j \leq v_k$.
2. $v_j < v_k$ (strictly increasing value).
3. $p_j < p_k$ (a rightward move).

The algorithm's inner loop iterates through all $j < k$ and checks conditions (2) and (3). By the IH, for any such valid j , $\text{EvenLen}[j]$ holds the optimal length. The recurrence $\text{OddLen}[k] \leftarrow \max(\dots, \text{EvenLen}[j] + 1)$ correctly finds the maximum over all valid predecessors. The base case of 1 is preserved.

A symmetric argument holds for $\text{EvenLen}[k]$. It requires a leftward move ($p_j > p_k$) from an odd-length subsequence. The recurrence correctly updates $\text{EvenLen}[k]$ based on the optimal values $\text{OddLen}[j]$ provided by the IH.

Thus, the claim holds for all k by the principle of mathematical induction. The final result is the maximum over all computed lengths, which is correct. $\square$

2.4 Complexity Analysis

- **Time Complexity:** $O(n^2)$. Creating and sorting P takes $O(n \log n)$. The nested loops run in $O(\sum_{k=1}^n (k-1)) = O(n^2)$, which is the dominant term.
- **Space Complexity:** $O(n)$. Storing P , OddLen , and EvenLen requires $O(n)$ space.

3 Iterative Dynamic Programming

3.1 Algorithm Description

This approach works directly on the original array indices using a 2D DP table, where $\text{dp}[i][0]$ stores the max odd-length LIBFS and $\text{dp}[i][1]$ stores the max even-length LIBFS ending at i . The governing recurrence relations are:

$$\begin{aligned} \text{dp}[i][0] &= \max \left(1, 1 + \max_{\substack{j < i \\ A[j] < A[i]}} \text{dp}[j][1] \right) \\ \text{dp}[i][1] &= \max \left(0, 1 + \max_{\substack{j > i \\ A[j] < A[i]}} \text{dp}[j][0] \right) \end{aligned}$$

A dependency conflict arises: computing $\text{dp}[i][1]$ requires values $\text{dp}[j][0]$ where $j > i$. This algorithm resolves this by iteratively applying the recurrences in passes. It repeatedly updates the entire DP table until the values converge and no further improvements can be made.

3.2 Pseudocode

3.3 Proof of Correctness

Proof. The algorithm processes elements in increasing order of value, which induces a topological ordering on all valid transitions: if $A[j] < A[i]$, then the pair $(A[j], j)$ precedes $(A[i], i)$ in P . Hence every valid predecessor j of i is finalized before i is processed.

Algorithm 2 Value-Sorted 2D Dynamic Programming ($O(n^2)$)

```
1: procedure SOLVEVALUESORTEDDP( $A, n$ )
2:   if  $n = 0$  then return 0
3:   end if
4:   Construct list  $P = \{(A[i], i)\}$  for  $i = 1 \dots n$ .
5:   Sort  $P$  by increasing value, breaking ties by smaller index.
6:   Initialize  $\text{dp}[1 \dots n][0 \dots 1]$ .
7:   for  $i \leftarrow 1$  to  $n$  do
8:      $\text{dp}[i][0] \leftarrow 1$  ▷ odd-length base case
9:      $\text{dp}[i][1] \leftarrow 0$  ▷ no even sequence yet
10:  end for
11:  for  $k \leftarrow 1$  to  $n$  do
12:     $(v_k, p_k) \leftarrow P[k]$ 
13:    for  $j \leftarrow 1$  to  $k - 1$  do
14:       $(v_j, p_j) \leftarrow P[j]$ 
15:      if  $v_j < v_k$  then
16:        if  $p_j < p_k$  then ▷ right move; odd subsequence
17:           $\text{dp}[p_k][0] \leftarrow \max(\text{dp}[p_k][0], \text{dp}[p_j][1] + 1)$ 
18:        else if  $p_j > p_k$  then ▷ left move; even subsequence
19:           $\text{dp}[p_k][1] \leftarrow \max(\text{dp}[p_k][1], \text{dp}[p_j][0] + 1)$ 
20:        end if
21:      end if
22:    end for
23:  end for
24:  return  $\max_{i,p} \text{dp}[i][p]$ 
25: end procedure
```

Claim. After completing iteration k of the outer loop, for the element (v_k, p_k) , both $\text{dp}[p_k][0]$ and $\text{dp}[p_k][1]$ store the exact lengths of the longest odd and even increasing back-and-forth subsequences ending at index p_k .

Base Case ($k = 1$). For the smallest value, no smaller predecessor exists, so $\text{dp}[p_1][0] = 1$ and $\text{dp}[p_1][1] = 0$, which is correct.

Inductive Step. Assume the claim holds for all $j < k$. To compute the lengths for (v_k, p_k) , the algorithm inspects every $j < k$ satisfying $v_j < v_k$.

- If $p_j < p_k$, a rightward move produces an odd-length extension, so $\text{dp}[p_k][0] = \max(\text{dp}[p_k][0], \text{dp}[p_j][1] + 1)$. Since $\text{dp}[p_j][1]$ is already optimal by the induction hypothesis, this yields the optimal odd subsequence ending at p_k .
- If $p_j > p_k$, a leftward move produces an even-length extension, so $\text{dp}[p_k][1] = \max(\text{dp}[p_k][1], \text{dp}[p_j][0] + 1)$. Again, $\text{dp}[p_j][0]$ is finalized, ensuring optimality.

Because all possible smaller-valued predecessors have been examined and all their DP states are finalized before iteration k , both recurrences yield globally optimal results for (v_k, p_k) . Thus, by induction, the claim holds for all k .

Finally, the answer $\max_{i,p} \text{dp}[i][p]$ equals the maximum LIBFS length over all possible endpoints and parities. $\square$

3.4 Complexity Analysis

- **Time Complexity:** $O(n^2)$. Two passes each take $O(n^2)$ time.
- **Space Complexity:** $O(n)$. The DP table requires $O(n \times 2) = O(n)$ space.

4 Recursive Approach with Memoization

4.1 Algorithm Description

A top-down dynamic programming approach can also be formulated. The state can be defined as (i, p) , representing the problem of finding the longest LIBFS ending at index i with a given parity p . A standard top-down approach is to define a function ‘solve(i, p)’ that directly computes $\text{dp}[i][p]$ by making recursive calls for its predecessors.

4.2 Pseudocode

4.3 Proof of Correctness

Proof. The proof is by strong induction on the subproblems, ordered by the number of recursive calls required to solve them. **Claim:** The function ‘Solve(i, p)’ correctly returns the length of the longest LIBFS ending at index i with parity p .

Base Cases: The recursion terminates at base cases where a subproblem finds no valid predecessors. For parity = ODD, the loop for $j < i$ might not find any valid predecessor, resulting in $\text{max_prev_len} = 0$ and a correct return value of 1. For parity = EVEN, if no predecessor is found, it correctly returns 0.

Inductive Hypothesis (IH): Assume for any subproblem (j, p') that requires fewer recursive calls to solve than (i, p) , ‘Solve(j, p)’ returns the correct value.

Inductive Step: When computing ‘Solve(i, p)’, the function makes recursive calls to ‘Solve(j, p)’ for its predecessors. These subproblems are necessarily simpler (as they form prefixes of the

Algorithm 3 Recursive DP with Memoization ($O(n^2)$)

```
1: procedure SOLVERECURSIVEDP( $A, n$ )
2:   Initialize memoization table  $\text{memo}[1 \dots n][0 \dots 1]$  with a sentinel value (e.g., -1).
3:    $\text{maxLength} \leftarrow 0$ 
4:   for  $i \leftarrow 1$  to  $n$  do
5:      $\text{maxLength} \leftarrow \max(\text{maxLength}, \text{Solve}(i, \text{ODD}), \text{Solve}(i, \text{EVEN}))$ 
6:   end for
7:   return  $\text{maxLength}$ 
8: end procedure

9: function SOLVE( $i, \text{parity}$ )
10:  if  $\text{memo}[i][\text{parity}]$  is not sentinel then return  $\text{memo}[i][\text{parity}]$ 
11:  end if
12:  if  $\text{parity} = \text{ODD}$  then ▷ Computing odd-length LIBFS ending at  $i$ 
13:     $\text{max\_prev\_len} \leftarrow 0$ 
14:    for  $j \leftarrow 1$  to  $i - 1$  do ▷ Requires right move from an EVEN sequence
15:      if  $A[j] < A[i]$  then
16:         $\text{max\_prev\_len} \leftarrow \max(\text{max\_prev\_len}, \text{Solve}(j, \text{EVEN}))$ 
17:      end if
18:    end for
19:     $\text{result} \leftarrow 1 + \text{max\_prev\_len}$ 
20:  else ▷ Computing even-length LIBFS ending at  $i$ 
21:     $\text{max\_prev\_len} \leftarrow 0$ 
22:    for  $j \leftarrow i + 1$  to  $n$  do ▷ Requires left move from an ODD sequence
23:      if  $A[j] < A[i]$  then
24:         $\text{max\_prev\_len} \leftarrow \max(\text{max\_prev\_len}, \text{Solve}(j, \text{ODD}))$ 
25:      end if
26:    end for
27:     $\text{result} \leftarrow 0$ 
28:    if  $\text{max\_prev\_len} > 0$  then  $\text{result} \leftarrow 1 + \text{max\_prev\_len}$ 
29:    end if
30:  end if
31:   $\text{memo}[i][\text{parity}] \leftarrow \text{result}$ 
32:  return  $\text{result}$ 
33: end function
```

path to i) and thus are covered by the IH. The function iterates through all valid predecessors for the given state (i, p) , retrieves their optimal lengths via the recursive calls (which are correct by IH), and takes the maximum. This correctly applies the Bellman equation for the longest path in the implicit state graph, ensuring the result for ‘Solve(i, p)’ is optimal. Due to memoization, each of the $2n$ subproblems is computed only once. $\square$

4.4 Complexity Analysis

- **Time Complexity:** $O(n^2)$. There are $2n$ states (subproblems). Each state computation involves a loop of up to $O(n)$ iterations. Since each state is computed once, the total time is $O(n) \times O(n) = O(n^2)$.
- **Space Complexity:** $O(n)$. The memoization table has $2n$ entries. The recursion depth is at most n .

5 Graph-Based Approach (Longest Path in a DAG)

5.1 Algorithm Description

This problem can be transformed into finding the longest path in a Directed Acyclic Graph (DAG).

- **Vertices:** $2n$ vertices, $v_{i,\text{odd}}$ and $v_{i,\text{even}}$ for each index i .
- **Edges:** For any i, j where $A[i] > A[j]$:
 1. Add edge $v_{j,\text{even}} \rightarrow v_{i,\text{odd}}$ if $j < i$ (right move).
 2. Add edge $v_{j,\text{odd}} \rightarrow v_{i,\text{even}}$ if $j > i$ (left move).

The problem becomes finding the longest path in this state-space graph, which is a standard DAG problem.

5.2 Pseudocode

5.3 Proof of Correctness

Proof. The proof relies on the correctness of the reduction to the longest path in a DAG problem.

Claim: There is a bijection (a one-to-one and onto mapping) ϕ from the set of all valid LIBFS to the set of all simple paths in the constructed graph G .

Let $I = (I_1, I_2, \dots, I_\ell)$ be a valid LIBFS. We map it to a path $P = (u_1, u_2, \dots, u_\ell)$ in G where $u_k = (I_k, \text{odd})$ if k is odd, and $u_k = (I_k, \text{even})$ if k is even. For any $k \in [1, \ell - 1]$, we must show an edge exists from u_k to u_{k+1} .

- If k is odd, $u_k = (I_k, \text{odd})$ and $u_{k+1} = (I_{k+1}, \text{even})$. Since I is a valid LIBFS, we know $I_{k+1} < I_k$ and $A[I_{k+1}] > A[I_k]$. By construction, an edge exists from $v_{I_k, \text{odd}}$ to $v_{I_{k+1}, \text{even}}$.
- If k is even, a symmetric argument shows an edge exists from $v_{I_k, \text{even}}$ to $v_{I_{k+1}, \text{odd}}$.

Thus, ϕ maps any LIBFS to a valid path. The mapping is injective (unique sequences map to unique paths) and surjective (any path in the graph can be mapped back to a valid LIBFS). The length of the LIBFS is the number of vertices in the path.

The graph G is a DAG because edges only go from a state corresponding to value $A[j]$ to a state for value $A[i]$ if $A[i] > A[j]$, making cycles impossible. The algorithm for finding the longest path in a DAG using topological sort and edge relaxation is known to be correct. Therefore, this approach correctly solves the problem. $\square$

Algorithm 4 Graph-Based Longest Path ($O(n^2)$)

```
1: procedure SOLVEGRAPH( $A, n$ )
2:   Phase 1: Graph Construction
3:   Initialize adjacency list ‘adj’ and in-degree map in_degree.
4:   for  $i \leftarrow 1$  to  $n$  do
5:     for  $j \leftarrow 1$  to  $n$  do
6:       if  $A[i] > A[j]$  then
7:         if  $j < i$  then
8:           Add edge from  $(j, \text{EVEN})$  to  $(i, \text{ODD})$ ; Increment in_degree $[(i, \text{ODD})]$ .
9:         else if  $j > i$  then
10:          Add edge from  $(j, \text{ODD})$  to  $(i, \text{EVEN})$ ; Increment in_degree $[(i, \text{EVEN})]$ .
11:        end if
12:      end if
13:    end for
14:  end for
15:  Phase 2: Longest Path Calculation
16:  Initialize map ‘lengths’; ‘lengths’ $[(i, \text{ODD})] \leftarrow 1$ , ‘lengths’ $[(i, \text{EVEN})] \leftarrow 0$ .
17:  Create queue ‘q’ and add all nodes with an in-degree of 0.
18:  Create list ‘topo_order’.
19:  while  $q$  is not empty do
20:     $u \leftarrow q.\text{dequeue}()$ . Add  $u$  to ‘topo_order’.
21:    for each neighbor  $v$  of  $u$  do
22:      Decrement in_degree $[v]$ .
23:      if in_degree $[v] = 0$  then  $q.\text{enqueue}(v)$ 
24:    end if
25:  end for
26: end while
27: for each node  $u$  in ‘topo_order’ do
28:   if ‘lengths’ $[u] \neq 0$  then
29:     for each neighbor  $v$  of  $u$  do
30:       ‘lengths’ $[v] \leftarrow \max(\text{‘lengths’}[v], \text{‘lengths’}[u] + 1)$ 
31:     end for
32:   end if
33: end for
34:   return maximum value in the ‘lengths’ map.
35: end procedure
```

Let $A[1..n]$ be an array of positive integers. An *increasing back-and-forth subsequence* is a sequence of indices $I[1..ℓ]$ satisfying the following properties:

1. $1 \leq I[j] \leq n$ for all j .
2. $A[I[j]] < A[I[j + 1]]$ for all $1 \leq j < ℓ$.
3. If j is even, then $I[j + 1] > I[j]$ (move right).
4. If j is odd, then $I[j + 1] < I[j]$ (move left).

In less formal terms, imagine a token placed on any square of the array. At each step, the token moves to a square containing a larger number. The direction of movement alternates: left when on an odd-indexed step, right when on an even-indexed step. The sequence of positions visited by the token forms an increasing back-and-forth subsequence.

Task:

Design an algorithm to compute the length of the longest increasing back-and-forth subsequence for a given array $A[1..n]$.

Figure 1: Enter Caption

5.4 Complexity Analysis

- **Time Complexity:** $O(n^2)$. The number of vertices $V = 2n$. Graph construction takes $O(n^2)$ and can create up to $E = O(n^2)$ edges. Both topological sort and the final DP pass take $O(V + E) = O(n + n^2) = O(n^2)$.
- **Space Complexity:** $O(n^2)$. Storing the graph's adjacency list can require $O(n^2)$ space in the worst case.

6 Paper Showing Notes

- The breakup is as follows:
 - 1. algorithm: 4 (2 for base case and transitions or algo setup, 2 for the remaining logic, possibly formalized as pseudocode)
 - 2. proof of correctness/optimalty: 4
 - 3. 1 for TC, 1 for SC
- Suboptimal solutions result in a max of 6 marks. 1 for algo desc, 2 for pseudocode, 2 for proof, 1 total for TC+SC
- Proofs of incorrect algorithms get zero marks. An algorithm is wrong if it doesn't pass one or more test cases (not considering time/space constraints, only correctness). In many cases, I've provided in my correction an example of such a failure case. Before coming to ask for marks in the question, you must show evidence of you actually dry running the test case provided in the corrected paper.
- Considering the question to ask for a subarray instead of a subsequence will result in an incorrect algorithm and 0 in the question as a whole. This is non-negotiable.
- The question clearly states to find the **LONGEST INCREASING BACK-AND-FORTH SUBSEQUENCE**. This problem was directly from your problem set, defiend as shown in an image below. The sequence is strictly monotonic. Only the indices alternate (go abck and forth) depending on which "step" we are currently on. Failure to recognize this will lead to an incorrect algorithm and a wrong answer. This merits a 0 on the question, but leniency has been exercised in some cases where understanding is demonstrated regardless.
- The **INDICES** move back and forth, as per the problem definition. Imagine a token placed on any square of the array. At each step, the token moves to a square containing a larger number. The **direction of movement alternates**: left when on an odd-indexed

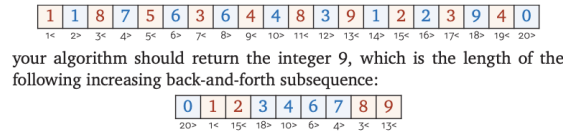


Figure 2: Example

step, right when on an even-indexed step. The **sequence of positions** visited by the token forms an increasing back-and-forth subsequence.

- Lack of initialization, vagueness, and lack of mathematical rigour will result in marks being deducted accordingly
- The "correct" time and space complexity for the wrong algorithm is the wrong time and space complexity. No marks will be awarded.
- I have glossed over silly errors and completed the pseudocode for you where it seems almost complete. Your algo is marked wrong **ONLY** when the fully realized version of it does not pass a test case.
- In some of your answer scripts, I may have written "suboptimal" or something along those lines. Feel free to ignore this entirely, you have not lost marks for it. If you feel you have, approach me.
- Some of you might find scratched out words, or a different marking scheme mentioned in your paper than what I've mentioned here. This is because I went over the papers for this questions twice, the second time being with a revised and more lenient marking scheme in an effort to bring up the average. The one mentioned here is the one we will adhere to.
- There exists an optimization that can bring down the time complexity to $O(N \log N)$, but I did not consider that to be the sole optimal solution. Only one person has done this, and if that is you, please do approach me, we have unfinished business.
- One of you has used a singular DP state representative of two sub-states. It has been marked zero for now owing to wrongness, but I am open to change that if you can successfully argue for its correctness.
- Around 3 of you used a sort of 2d-adjacent DP solution with $dp[ind][prev][flag]$. I believe I missed **one** such solution and gave it zero by accident. If this is you, kindly do approach me. For the rest, I've marked your solution fairly, even ignoring the lack of mathematical correctness.
- A **LOT** of you have given me the classic 2D dp solution, the transitions and states for which you can see under the "Dynamic Programming with Sorted Input" solution. Note that this algorithm works only if you iterate through your array **SORTED** by value keeping track of the indices. However, seeing as this would absolutely devastate most of your answers, again, we have chosen to gloss over this technicality as well and award the understanding of the transitions and the optimal substructures.